

EE 314: Digital Circuits Laboratory

Term Project Description

FPGA-Based 2-Player Fighting Game

Please review the Academic Integrity Guide for Students. Taking any codes from any source not given in ODTUClass is strictly prohibited. **Any questions regarding the project should be asked through the project forum on ODTUClass.**

This project involves the design and implementation of a minimalistic yet engaging fighting game for two players, built entirely in hardware using Verilog HDL and deployed on an FPGA development board.

The game is inspired by Footsies, an open-source fighting game developed by HiFight - detailed information about the game mechanics and design can be found on the [official website](#) [1]. To gain a deeper understanding of game mechanics and terminology, it is strongly recommended to play the original game. The game includes helpful features such as frame-by-frame play feature and debug visualizations (e.g., hitbox overlays), which can aid in analyzing gameplay and understanding timing and interactions more precisely.

In the game, each player controls a character who can move within a 2D arena and perform attacks with the goal of reducing the opponent's health to zero. The game will run in real time on an FPGA with output on a VGA display and will feature physical input via a keypad.

The goal is to reinforce students' understanding of digital design principles through a complete system that involves real-time graphics, user input handling, finite state machines (FSMs) or algorithmic state machines (ASMs), and interactive gameplay. This project serves as a hands-on exercise in integrating multiple digital subsystems and debugging complex behavior in hardware.

- The FPGAs will be provided by the laboratory. For details of borrowing an FPGA for your group, refer to the [FPGA Borrowing](#) document on ODTUClass.
- For player input, you will need an external keypad that is compatible with the FPGAs' peripherals (GPIO pins), which you need to purchase yourselves. If you have an issue finding a suitable keypad, you may contact the coordinator assistant.
- Early demonstrations are possible and encouraged. Please contact the coordinator assistant in advance for scheduling.

Important Dates, Deliverables & Grading

- Report: 20th June (**25% Credits**)
- Project Codes: 20th June (**Required for the demonstration**)
- Project Video: 20th June (**5% Credits**)
- Demonstration: 21th–23th June (**70% Credits**)

For the details on deliverables, please refer to the [Term Project Deliverables](#) document on ODTUClass.

1 Game Concept Overview

The game is a side-view 2D fighting experience for two human players. Player(s) use physical buttons on a keypad to control their respective characters. Each character can perform basic actions such as:

- Move Left / Right
- Attack
- Special Attack
- Block (Defensive Mechanic)

Matches are played as first to 3, meaning that the first player to win 3 rounds wins the match.

Players start each round with 3 block points. A hit detection system calculates whether an attack successfully hits, whiffs, or is blocked. The possible outcomes for an attack are:

- Whiffs, meaning the hitbox **does not collide** with opponents hurtbox. Nothing further happens in this case.
- Hits the opponent, meaning the hitbox **collide** with the opponents hurtbox while the opponent **was not** blocking. If it was a basic attack opponent enters the hit stun state, if it was a special attack the opponent is KO'ed (Knocked Out) and the attacking player wins the round.
- Opponent blocks the attack, meaning the hitbox **collide** with the opponents hurtbox while the opponent **was** blocking. If an attack is blocked while no block points are available the opponent enters the Guard Break state otherwise they enter block stun state.

Note that, if an attack is blocked or hits, the opponent's block counter is reduced. The player that wins 3 rounds wins the match.

2 Game Flow and System Behaviour

2.1 Game Start and Initialization

The game enters an idle state upon power-up or reset. During this time, the players with two different button sets will be displayed on the VGA display named P1 and P2. Then, the players can switch between the left and right sides (to decide which player controls the character on which side) using their controllers. Seven-Segment Displays must also show "P1" and "P2" on their current sides. The game starts when the designated Player 1 presses the confirm button (You can pick any of the buttons assigned to Player 1) and both players have picked opposing sides (There can't be two players on the left/right or a player with no side). A countdown like "3", "2", "1", "START" is first displayed on the screen, during which no input should be accepted. After the countdown ends, the fighting gameplay can begin.

2.2 Gameplay Loop

The core of the game is a deterministic state machine that evaluates player inputs and transitions characters between states based on rules defined for movement and attacks. **The character can only be in one state at a given point in time.** Game logic runs with a 60 Hz clock. Since the VGA refresh rate is also 60 Hz, the character state changes occur solely between consecutive frames.

Mandatory Debug Mode: You should also use SW[1] to switch between a 60Hz signal and a KEY on the FPGA for the clock of Game Logic. This will be used during your demonstrations and is very helpful for you when you are debugging.

Character States:

- **IDLE:** No input detected, character is standing still.
- **MOVING FORWARD:** Horizontal position changes towards the enemy by **three pixels per frame** on the 60 Hz VGA display. Note that the players cannot cross each other.

- **MOVING BACKWARDS:** Horizontal position changes away from the enemy by **two pixels per frame** on the 60 Hz VGA display. Note that the players cannot move out of the screen boundaries.
- **ATTACK (Basic):** A three-phase attack consisting of startup, active, and recovery phases.
- **ATTACK (Special):** A three-phase attack that also moves the player forward. This special attack can be executed in two ways:
 1. by pressing the attack button during the recovery phase of a successful basic attack (i.e., one that does not whiff), or
 2. by holding the attack button to charge and releasing it after a set duration.

You must implement both ways to get full credit. You can decide on how long the button should be held to "charge" the special attack, and how much the special attack moves the player forward. Note that the chosen distance and charge duration should be discernible by human eye.

- **HITSTUN:** Temporary state after being hit.
- **BLOCKSTUN:** Temporary state after blocking.
- **GUARD BREAK:** Temporary state after blocking with no block points.

Both attack actions are committal, meaning once the attack starts with the startup, regardless of the user inputs, it should go through all the stages*. Note that if the user gets hit during their attack, they enter hitstun, which would cancel the rest of their attack stages.

That is, any combination of buttons pressed after the attack starts has no effect, except for canceling into the special attack from the basic attack.

***Cancel into special attack:** Happens when a player presses the attack button once more during the recovery frames of a basic attack that did not whiff. This input cancels the remaining recovery frames and transitions immediately into the startup phase of the special attack.

Check the Table 1 on the Appendix A for attack, hitstun, blockstun and guard break durations. Also, check the Appendix A for example cases on attack, hurtbox, and hitbox interactions.

2.3 Hit Detection and Resolution

In fighting games and many other types of competitive action games, characters are not just represented visually by their sprites or 3D models — they are also defined by invisible zones known as hitboxes and hurtboxes. These are fundamental concepts in game mechanics and collision detection.

- **Hurtbox:** A hurtbox is an invisible area that defines the region of a character's body that can be damaged by an opponent's attack. In essence, it's the "vulnerable" area of a character. Hurtboxes typically follow the character closely, changing position as the character moves.
- **Hitbox:** A hitbox is the active part of an attack — the area that can inflict damage or trigger a reaction when it intersects with an opponent's hurtbox. Hitboxes are generated when a player performs an attack action. Just like hurtboxes, hitboxes are usually not static; they follow the attack animation and may only appear for a few frames (fractions of a second) during the move's active phase. It is important to note that during the recovery frames of an attack, the hitbox extended from the character is converted into a hurtbox. Meaning the extended part is vulnerable to be hit by the opponent during the recovery.
- **Interaction Between Hitbox and Hurtbox:** The core mechanic revolves around the interaction between hitboxes and hurtboxes. When the hitbox of an attacking character overlaps (or "collides") with the hurtbox of the defending character, the attack is considered to have "connected" or "landed." This results in the defending character entering the hitstun state (if it was a basic attack) or losing the round (if it was a special attack). If the defending character was blocking,

they would enter the blockstun state instead if they have a block point or the Guard Break state if they have none. In all cases the defending character decreases their block counter.

- **Visual Debugging Aids:** While normally invisible during gameplay, developers often visualize hitboxes and hurtboxes in development tools or debug modes for testing and balancing purposes. In many visualizations:

- Hurtboxes are often shown in yellow, indicating where the character can be hit.
- Hitboxes are commonly represented in red, showing the active attack zone.

These visual aids are crucial for debugging the design.

Blocking is done by walking backwards. Meaning, if an attack lands on an opponent walking backwards, the opponent counts as having blocked the attack instead of getting hit. There should **not** be any other way to block attacks, such as a dedicated button.

It is important to understand that a properly functioning hit detection system is fundamental to the game's logic. Without reliable hit detection, it is not feasible to implement key mechanics such as blocking, hitstun, blockstun, guard break, game-over conditions, and related game states in a coherent and consistent manner.

Therefore, the development process must follow a logical sequence. Certain components—such as attacking and hit detection—must be implemented and verified before progressing to more advanced gameplay features.

Edge case, simultaneous hits: If both characters enter the active attack phase at the same time and their hitboxes overlap, both are hit. Meaning they both enter hitsun and lose a block point if it was a basic attack, round resets in a draw if it was a special attack.

2.4 Game Over and Rounds

When a player is KO'ed, the round resets from the initial state with something like the "3", "2", "1", "Start". When a player wins 3 rounds, the system enters a "Game Over" state. A message is displayed on the screen declaring the winner. At this point:

- The seven-segment display shows the winner and the score.
- LEDs blink to indicate the end of the game.
- Player 1 can exit to the menu using their buttons.

3 Input and Output Specification

3.1 Input Devices

All player controls should be implemented using button inputs. Ideally, two separate keypads should be used—one for each player. Player 1 may use the onboard FPGA buttons, while Player 2 can use an external keypad as an example (any basic functional keypad is acceptable). You should implement a debouncing scheme to filter out transient fluctuations due to mechanical bouncing.

Each player must have exactly three buttons assigned to the following actions:

- Move Left
- Move Right
- Attack

No additional control inputs should be used beyond these three.

The players can press none or any combination of buttons. The players can also hold the button down, for charging their special attack and if charged for a sufficient duration when they release the button the special attack should trigger. Note that for attacking you are checking for the press of the button

(That is the transition of the button from unpressed to pressed, i.e. edge detection). If a player holds the button down the character should not continuously attack.

The game should have a deterministic input handling such that the same case occurs for all button combinations. For example, holding both Left and Right simultaneously should result in predictable and deterministic behaviour for both players.

3.2 Output Devices

3.2.1 VGA Specification

The VGA controller operates with a 25.172 MHz clock but a 25 MHz clock should also do the trick. The clock on our FPGA boards runs at 50 MHz, so you should divide it accordingly. **You are not required to write your own VGA controller.** This [website](#) [2] has a fully functional VGA controller you can use and is a great starting point, but you can use other sources if you want. Recall that whichever source you use, you should properly reference it in the report.

Drive the VGA with:

- Resolution: 640×480 at 60Hz.
- Color Format: 8-bit (3-3-2 RGB).

Menu Screen

- Display a menu screen. The minimum expected menu screen should at least display the text “MENU” and which player has selected which side (left/right) at all times. That said, you can get as creative as you want.

Gameplay Screen

- Display a countdown: “3”, “2”, “1”, “START” (or something similar) before the match begins.
- Character sprites should be rendered at a resolution of **64×240 pixels**.
- The players and their actions should be displayed on the VGA screen. Some of these actions (moving and attacking) must be animated on the screen. While the others (getting hit, blockstun, hitstun, attack phases, etc.) must be shown. For the attack phases you can just use colors or you can animate these as well, as long as they are discernible.
- Your game should have a background. It can be a static color or a static/moving pattern. A pure black background is not allowed.
- Number of rounds won for each player should be visible on the screen.
- Remaining block points for each player should be visible on the screen

Game Over Screen

- Freeze the players.
- Display the outcome of the match on the screen.
- The game over screen should be present until Player 1 presses any button. Then, the display should return to the menu screen.

3.2.2 Seven-Segment Displays

Menu Mode

- On 7-segment displays, “P1” and “P2” is written in their respective sides (Player on the left is written on the left of the 7-segments). This should change as players change their sides.

Gameplay Mode

- After the game mode is selected, the 7-segment display should show "P1 VS P2" or "P2 VS P1" depending on whose on which side.
- The 7-segment display should not change during entire gameplay.

Game Over Mode

- Display the result in the format "**P1-3-X**" or "**P2-3-X**", where the X number represents the rounds won by the losing player. For example, if the P1 won 3 rounds while P2 won 1, it should display "P1-3-1"

The game over display should be present until Player 1 presses any button. Then, the 7-segment display should return to the menu mode.

3.2.3 LED Indicators

Menu Mode

- All LEDs remain **off**.

Gameplay Mode

- The **leftmost 3 LEDs** indicate **Left Player's won rounds**.
- The **rightmost 3 LEDs** indicate **Right Player's won rounds**.
- LEDs are turned on progressively to reflect number of rounds won.

Game Over Mode

- **All LEDs blink** simultaneously to indicate the end of the game.

References

- [1] HiFight, "FOOTSIES," 2018. Accessed: 2026-05-3.
- [2] V. H. Adams, "Vga driver in verilog," 2025. Accessed: 2026-05-3.

A Frame Data Information and Visuals

The special attack has a longer range compared to the basic attack, but suffers from a longer startup and recovery, meaning it requires more frames after the input is made to become active and yet more frames to recover. In the table recovery being negative means that the defender recovers that many frames faster than the attacker. Positive means that the attacker recovers faster than the defender.

For example for the basic attack:

- On hit, the attacker recovers 1 frame **later** than the defender.
- On block, the attacker recovers 3 frames **later** than the defender.
- On guard break, the attacker recovers 17 frames **earlier** than the defender.

Frame disadvantage means that after the attack connects, whether it lands or is blocked, the defender recovers sooner and can act before the attacker. As a result, if both players attempt to retaliate by pressing the attack button immediately after the interaction, the defender's follow-up move will start earlier. This can lead to the defender successfully interrupting the attacker's next move and scoring a hit.

Notice that if a guard break occurs from the basic attack and the attacker choses to cancel into the special attack, the special attack is guaranteed to hit as the Guard Break stun (17 frames) is larger than the special attack startup (14 frames) making it a KO.

For the upcoming figures, the character on the left side of the screen is designated as Player 1, while the character on the right side is designated as Player 2. Recovery values in Table 1 are given relative to the Attacker. Hence, -1 means the Attacker recovers 1 frame later than the receiver.

Note that this assumes the attack connects at the first active frame. If the attack connects at let's say the second frame (although unlikely can happen by the oppoent walking into the attack at it's second frame), recovery would be $+0$ since the attack has 16 more frames to conclude after the second active frame as opposed to 17 frames. That is to say, the blockstun/hitstun has a static duration.

Table 1: Frame Data for Attacks (Images taken from [1])

Move	Command	Frame			Recovery		
		Startup	Active	Recovery	On Hit	On Block	Guard Break
	Attack	5	2	17	-1	-3	+17
	Hold & Release Attack Attack → Attack	14	2	31	KO	-12	+3

You can see an example hurtbox/hitbox structure in Figure 5 taken from [1]. Note that in our case the hitbox and hurtbox of the attacks do not shrink or grow with each frame, they are immediately put out and after the recovery ends immediately vanished.

Suppose Player 1 performs an attack (which has a red hitbox) and Player 2 is standing nearby with a visible yellow hurtbox. If the attack hitbox reaches into Player 2's hurtbox during the active frames of the attack and Player 2 was not blocking, the game will register a hit, causing Player 2 to react accordingly (e.g., reduce block counter and enter hitstun state). This example is shown in Figure 1. If Player 2 blocks the attack, their block counter will still decrease, and Player 2 will enter the blockstun state. This example is shown in Figure 2. If the Player 2 blocks the attack with no block points he will enter the Guard Break state as shown in Figure 3.

Suppose Player 1 performs an attack (which has a red hitbox) and Player 2 is standing nearby with a visible yellow hurtbox. If the attack hitbox does not reach into Player 2's hurtbox during the active frames of the attack (meaning it whiffs), Player 2 can attack and hit the extended hurtbox of Player 1. This example is shown in Figure 4.

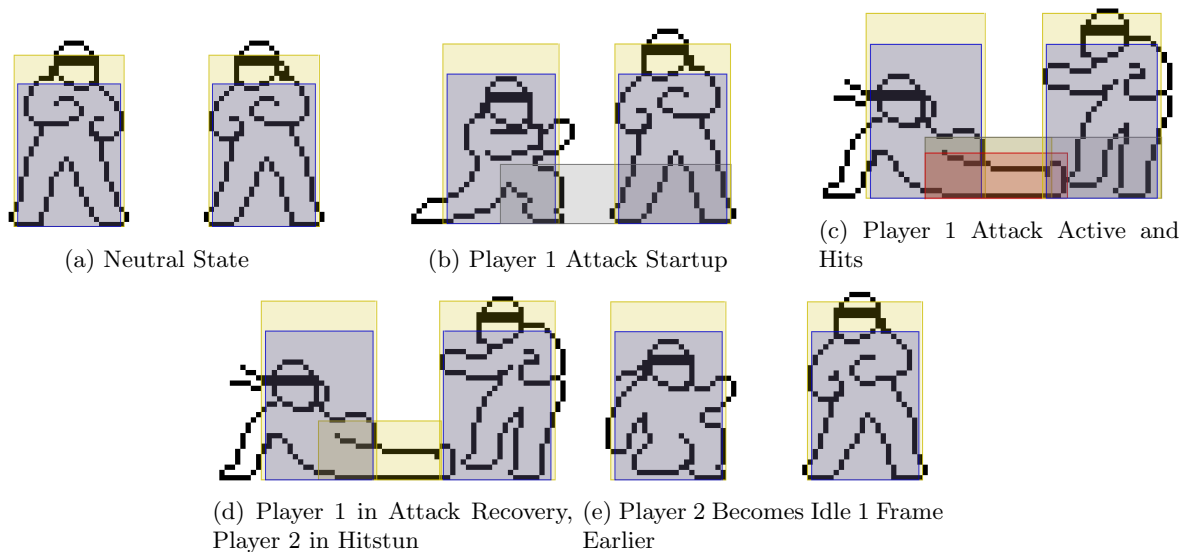


Figure 1: How a Hit Occurs (Images taken from [1])

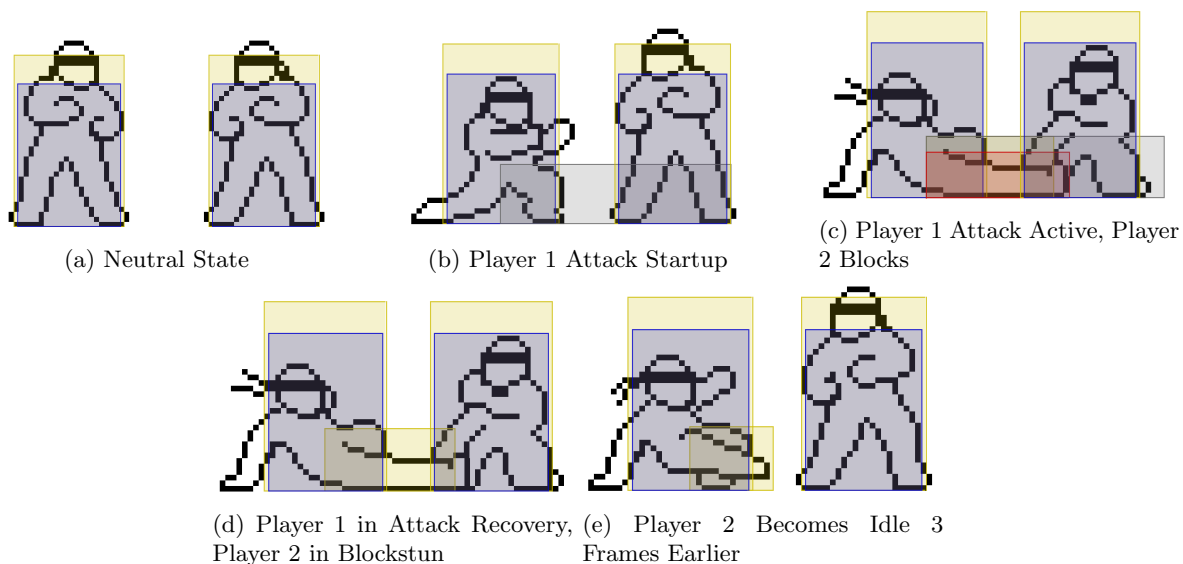


Figure 2: How a Block Occurs (Images taken from [1])

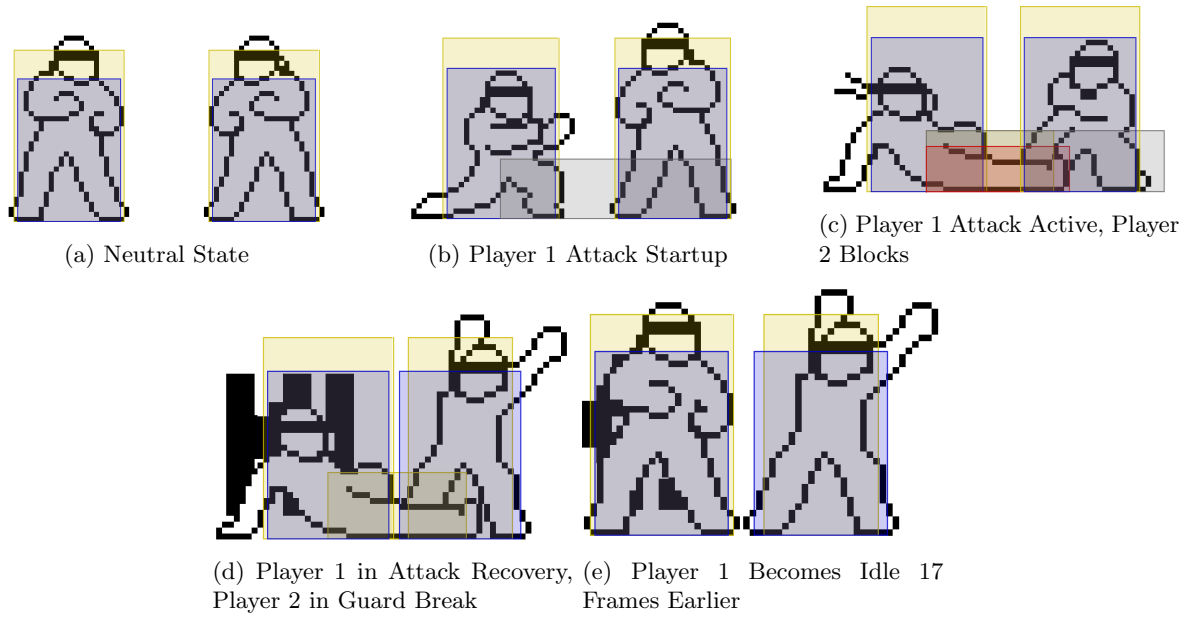


Figure 3: How a Guard Break Occurs (Images taken from [1])

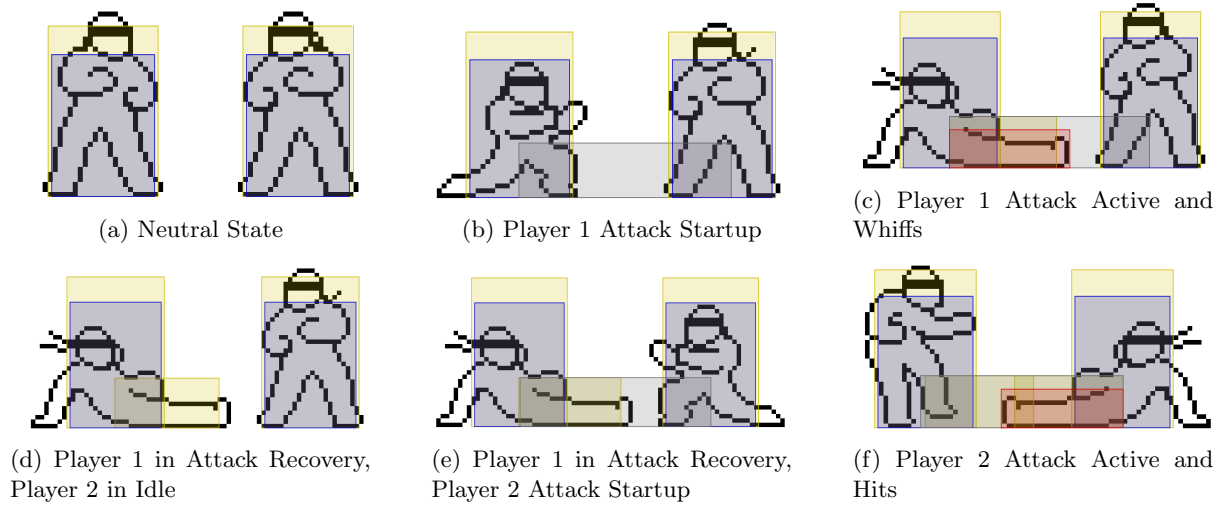
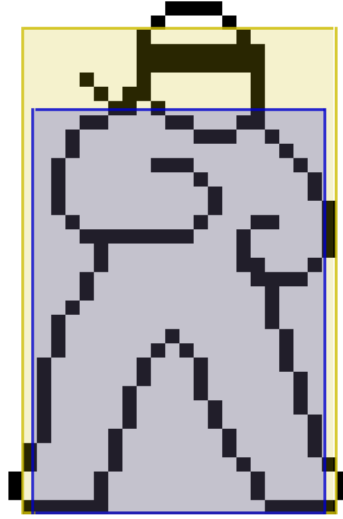
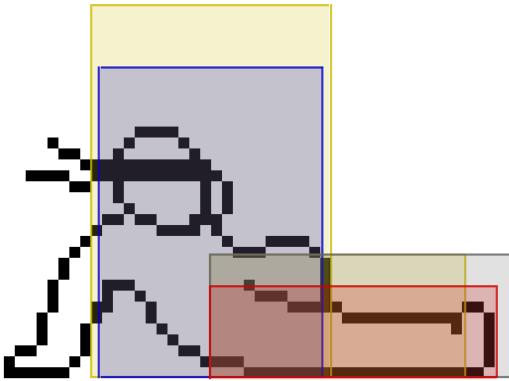


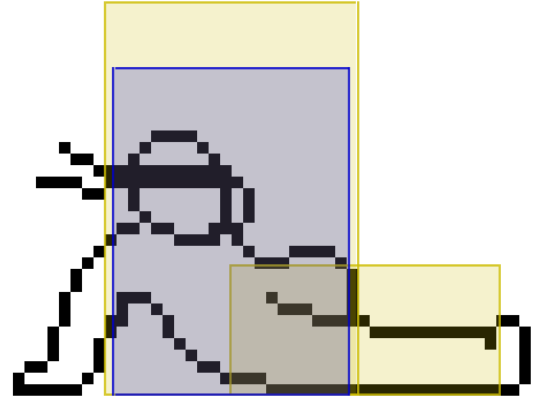
Figure 4: How a Whiff and Punish Occurs (Images taken from [1])



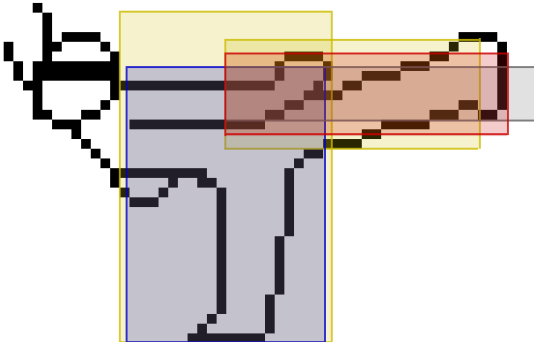
(a) Idle State Hurtbox



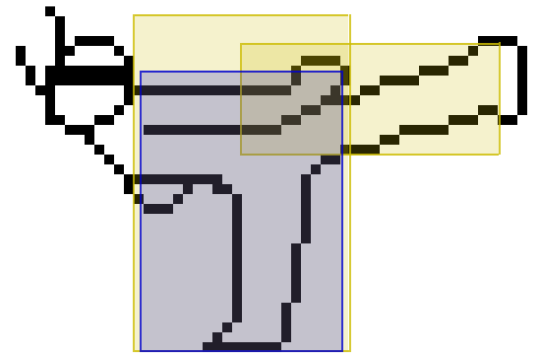
(b) Neutral Attack Hitbox



(c) Neutral Attack Recovery Hurtbox



(d) Special Attack Hitbox



(e) Special Attack Recovery Hurtbox

Figure 5: Hitbox and Hurtboxes of Attacks (Images taken from [1])

B Parts List

DE1-SoC Board

A VGA Monitor

Keypad(s) with at least 3 buttons

C Changelog